



Universidad Cenfotec
Proyecto de Ingeniería de Software 3

Plan General del Proyecto



COFFE & COMMITS

Estudiantes

Dereck Chavarría Mata
Matías Calvo Echeverría
Camilo Céspedes Umaña
Derek Carmiol Achio
Marcos Morales Céspedes
Gabriel Valverde Monge

Profesores

Nelson Allan Araya Alvarado
Jessica Cerdas Álvarez
Gino Alonso Marín León

Fecha de Entrega: Domingo 7 de Junio

Periodo Lectivo: II Cuatrimestre 2026

Audiencia

El presente documento está dirigido a todas las personas que participan o tienen interés en el desarrollo del proyecto **Brakket**. Su propósito es servir como guía para la planificación, organización y seguimiento de las actividades que se llevarán a cabo durante el desarrollo de la plataforma.

En primer lugar, está orientado al equipo de trabajo, ya que reúne los procesos, estándares, lineamientos y estrategias que se utilizarán para desarrollar el proyecto de manera ordenada y coordinada. De esta forma, todos los integrantes tendrán una referencia común sobre cómo se gestionará la calidad, los riesgos, la configuración del software y la planificación de las tareas.

Asimismo, el documento está dirigido a los profesores y facilitadores del curso Proyecto de Ingeniería de Software 3 de la Universidad Cenfotec, quienes podrán dar seguimiento al progreso del proyecto y evaluar la aplicación de las buenas prácticas de ingeniería de software utilizadas por el equipo.

Por otra parte, este documento también puede ser consultado por personas interesadas en la solución propuesta, como potenciales usuarios, organizaciones relacionadas con los esports o cualquier interesado en conocer el alcance, la estructura y la planificación general del proyecto.

En resumen, la audiencia principal de este documento está conformada por:

- El equipo de desarrollo de **Brakket**, incluyendo los coordinadores de calidad, desarrollo, soporte y gestión del proyecto.
- Los profesores, facilitadores y evaluadores académicos son responsables de dar seguimiento y revisar el proyecto.
- Personas, organizaciones, comunidades o futuros usuarios interesados en conocer la planificación, organización y alcance de la plataforma, particularmente dentro del ecosistema de los deportes electrónicos (esports).

Tabla de Contenidos

Introducción.....	4
Propósito del Documento.....	4
Descripción General del Sistema.....	4
Estructura del Documento.....	4
Definiciones, acrónimos y abreviaturas.....	6
Plan de Administración de la Calidad.....	8
Formato y Detalle de las Listas de Verificación.....	9
Diseño y Especificación de los Casos de Prueba.....	9
Herramienta de Gestión.....	10
Jerarquía de Pruebas.....	10
Plantilla de Caso de Prueba.....	11
Recopilación de Evidencias.....	12
Responsables de Ejecución y Aprobación.....	12
Diseño y Especificaciones de las Pruebas Unitarias.....	13
Diseño y Especificaciones de las Pruebas de Rendimiento.....	15
Definition of Done.....	18
Checklist de Definition of Done.....	18
Acuerdos Generales del Equipo.....	19
Plan de Gestión de Riesgos.....	20
Tabla de Riesgos.....	20
Tabla de Probabilidad e Impacto.....	22
Mitigaciones y Contingencias.....	23
Plan de Administración de la Configuración.....	25
Descripción de las Herramientas.....	25
Git.....	25
GitHub.....	25
GitHub Actions.....	25
Google Drive.....	25
Administración del Repositorio.....	25
Estructura del Repositorio.....	25
Estrategia de Ramas.....	26
Flujo de Trabajo.....	26
Reglas del Repositorio.....	26
Reglas Generales.....	26
Convención de Commits.....	26
Frecuencia de Actualización.....	27
Contenido de los Commits.....	27
Revisiones de Código.....	27
Plan de Estándares.....	28

Estándares de Interfaz Gráfica.....	28
Paleta de Colores.....	28
Tipografía.....	29
Nomenclatura de Controles.....	29
Componentes de la Interfaz.....	30
Estándares de Codificación.....	31
Estándares Generales.....	31
Estándares de Base de Datos.....	32
Estándares de Código en Java (Backend).....	32
Estándares de Código en Angular y Javascript (Frontend).....	33
Estándares de Documentación.....	33
Plan de Administración del Tiempo.....	34
Estructura de desglose de trabajo (WBS).....	34
Diagrama de Red.....	35
Herramienta de Gestión de Proyectos.....	36
Lista de Verificación.....	37

Introducción

Propósito del Documento

El presente documento tiene como propósito establecer el Plan General del Proyecto **Brakket**, definiendo los lineamientos que regirán la administración, control, seguimiento y ejecución del desarrollo de la plataforma. Este documento servirá como guía para todos los integrantes del equipo de trabajo, permitiendo coordinar adecuadamente las actividades relacionadas con la calidad, la gestión de riesgos, la administración de la configuración, la definición de estándares de desarrollo y la planificación del tiempo.

Descripción General del Sistema

Brakket es una plataforma web para la gestión integral de ligas y torneos de videojuegos competitivos. La solución permite administrar competencias desde su creación hasta su finalización, incorporando además funcionalidades de transmisión mediante Twitch, análisis de audiencia, gestión de patrocinadores, seguimiento estadístico y control administrativo.

La plataforma busca modernizar la administración de competencias esports mediante la automatización de procesos, la centralización de la información y la incorporación de herramientas tecnológicas que faciliten la toma de decisiones basada en datos.

Estructura del Documento

El documento se encuentra organizado en los siguientes apartados:

- **Definiciones, acrónimos y abreviaturas:** recopila los términos técnicos, siglas y conceptos clave utilizados a lo largo del documento, con el fin de garantizar una comprensión común.
- **Plan de Administración de la Calidad:** describe los mecanismos de control de calidad de los documentos y del producto, incluyendo listas de verificación, casos de prueba, pruebas unitarias, pruebas de rendimiento y la definición de terminado (Definition of Done).

- **Plan de Gestión de Riesgos:** identifica los riesgos del proyecto y define las tablas de impacto y probabilidad, así como las acciones de mitigación y contingencia asociadas.
- **Plan de Administración de la Configuración:** establece las herramientas, la estructura del repositorio y las reglas de control de versiones que el equipo utilizará para gestionar el software y la documentación.
- **Plan de Estándares:** define los estándares de interfaz gráfica y de codificación que se seguirán durante el desarrollo del proyecto.
- **Plan de Administración del Tiempo:** detalla la estructura de desglose de trabajo (WBS), el diagrama de red y las herramientas de gestión que se utilizarán para planificar y dar seguimiento a las tareas.

Cada apartado describe los procesos, herramientas y lineamientos que el equipo utilizará durante el desarrollo del proyecto.

Definiciones, acrónimos y abreviaturas

Término / Acrónimo	Definición
API	Application Programming Interface. Conjunto de reglas y protocolos que permiten la comunicación entre diferentes aplicaciones o servicios.
IA	Inteligencia Artificial. Tecnología utilizada para analizar información y realizar tareas que normalmente requieren inteligencia humana.
Twitch	Plataforma de transmisión en vivo utilizada para la difusión de contenido relacionado con videojuegos, deportes electrónicos y entretenimiento.
Esports	Deportes electrónicos o competiciones organizadas de videojuegos a nivel competitivo.
Sprint	Período de trabajo definido dentro de una metodología ágil en el que se desarrolla un conjunto específico de funcionalidades.
Scrum	Marco de trabajo ágil utilizado para la gestión y desarrollo de proyectos de software.
Product Backlog	Lista priorizada de requerimientos, funcionalidades y mejoras pendientes dentro del proyecto.
User Story	Descripción breve de una necesidad o funcionalidad desde la perspectiva de un usuario del sistema.
DoD	Definition of Done. Conjunto de criterios que deben cumplirse para considerar una tarea finalizada.
Git	Sistema de control de versiones distribuido utilizado para gestionar cambios en el código fuente.
GitHub	Plataforma basada en Git que permite almacenar repositorios y colaborar en equipo.
Pull Request	Solicitud para integrar cambios realizados en una rama hacia otra rama principal.
CI/CD	Continuous Integration / Continuous Deployment. Automatización de integración y despliegue.

Fixture	Calendario o programación de enfrentamientos entre participantes de un torneo.
Bracket	Representación gráfica de los cruces y resultados de una competición.
Test Case	Caso de prueba individual diseñado para validar una funcionalidad específica.
Test Cycle	Conjunto de casos de prueba agrupados para ejecutarse durante una iteración.
Test Plan	Documento o conjunto de ciclos de prueba que define la estrategia general de validación.
Unit Test	Prueba unitaria destinada a verificar el correcto funcionamiento de una unidad específica de código.
Performance Test	Prueba orientada a medir rendimiento, estabilidad y capacidad de respuesta.
UI	User Interface. Conjunto de elementos visuales con los que interactúa el usuario.
UX	User Experience. Experiencia general que percibe un usuario al utilizar el sistema.
ERS	Especificación de Requerimientos de Software.
MVP	Minimum Viable Product. Versión funcional mínima del producto.
Stakeholder	Persona, grupo u organización que tiene interés o participación en el proyecto.

Plan de Administración de la Calidad

La calidad representa uno de los pilares fundamentales para el éxito del proyecto **Brakket**, ya que no solo se busca desarrollar una plataforma funcional, sino también una solución confiable, estable, segura y capaz de satisfacer las necesidades de los distintos usuarios que interactuarán con ella. Debido a la naturaleza del sistema, que involucra la gestión de torneos, equipos, jugadores, resultados, transmisiones y estadísticas, resulta indispensable establecer mecanismos que permitan verificar continuamente que cada componente desarrollado cumple con los requisitos definidos y mantiene un nivel adecuado de calidad.

La administración de la calidad será un proceso continuo que acompañará todas las etapas del proyecto, desde la elaboración de la documentación inicial hasta la implementación y validación de las funcionalidades finales. El objetivo principal es prevenir errores antes de que lleguen a las fases finales del desarrollo, reducir la necesidad de retrabajos y garantizar que el producto entregado cumpla con los estándares acordados por el equipo.

Para lograrlo, el equipo implementará diversas actividades de aseguramiento y control de calidad. Entre ellas se encuentran la revisión de los documentos del proyecto mediante listas de verificación, la validación de requerimientos y criterios de aceptación, la ejecución de pruebas funcionales sobre las historias de usuario desarrolladas, la creación y ejecución de pruebas unitarias para validar el comportamiento del código, y la realización de pruebas de rendimiento que permitan medir la capacidad del sistema bajo diferentes niveles de carga.

Asimismo, la calidad no será responsabilidad exclusiva de los coordinadores de calidad, sino una responsabilidad compartida por todos los integrantes del equipo. Cada desarrollador deberá verificar que su trabajo cumpla con los estándares definidos antes de integrarlo al proyecto, mientras que los encargados de calidad supervisarán y validarán que los procesos establecidos se estén cumpliendo correctamente.

Como parte de esta estrategia, se utilizarán herramientas de apoyo para la gestión de pruebas, el control de versiones y el seguimiento de incidencias, permitiendo mantener evidencia de las revisiones realizadas y asegurar la trazabilidad de los cambios efectuados durante el desarrollo. Esto facilitará la detección temprana de problemas y permitirá tomar acciones correctivas de manera oportuna.

En términos generales, el Plan de Administración de la Calidad tiene como propósito garantizar que cada entregable generado durante el proyecto, tanto documental como de software, cumpla con los criterios de calidad definidos por el equipo y por el curso. De esta manera, se busca entregar una plataforma robusta, mantenible y alineada con los objetivos planteados para **Brakket**, minimizando la presencia de defectos y asegurando una experiencia satisfactoria para sus futuros usuarios.

Formato y Detalle de las Listas de Verificación

Como parte del proceso de aseguramiento de la calidad, cada entregable documental del proyecto deberá ser revisado antes de su entrega oficial mediante una lista de verificación. Estas listas permiten validar que el documento cumple con los requisitos establecidos por el curso, mantiene una estructura adecuada y presenta información consistente, completa y libre de errores significativos.

La revisión será realizada por al menos un integrante distinto al autor principal del documento, con el objetivo de asegurar una evaluación objetiva y detectar posibles inconsistencias antes de la entrega.

Cada lista de verificación incluirá los siguientes campos:

Elemento a Revisar	Cumple	No Cumple	Fecha de Revisión
--------------------	--------	-----------	-------------------

Diseño y Especificación de los Casos de Prueba

La validación funcional del sistema **Brakket** se realizará a través de casos de prueba formales, estructurados de manera jerárquica y gestionados mediante Jira. Cada requerimiento del backlog deberá estar respaldado por al menos un caso de prueba que valide sus criterios de aceptación antes de poder ser marcado como completado.

Herramienta de Gestión

La herramienta oficial para la gestión de pruebas funcionales será Jira, aprovechando su módulo de pruebas (Jira Test Management o integración con Zephyr Scale). Cada caso de prueba quedará vinculado a la historia de usuario correspondiente, permitiendo trazabilidad completa entre requerimientos y resultados de validación.

Jerarquía de Pruebas

La organización de las pruebas seguirá la siguiente jerarquía:

Test Plan → Test Cycle → Test Case

- Test Case: prueba individual y específica que valida un criterio de aceptación concreto.
- Test Cycle: conjunto de test cases agrupados por sprint o iteración de desarrollo.
- Test Plan: agrupación de test cycles con un propósito específico (funcional, regresión, humo).

La tabla siguiente ilustra cómo se organizan los planes y ciclos de prueba en el contexto de los sprints del proyecto:

Test Plan	Test Cycle	Test Case	Propósito
Functional Testing	Sprint 1 – Autenticación y Perfiles	TC-001 a TC-010	Validar autenticación con Google y gestión de perfiles
Functional Testing	Sprint 2 – Ligas y Torneos	TC-011 a TC-025	Validar creación, configuración e inscripción a torneos

Functional Testing	Sprint 3 – Fixtures y Resultados	TC-026 a TC-040	Validar generación de brackets y reporte de resultados
Regression Testing	Regresión Sprint 3	TC seleccionados de sprints anteriores	Verificar que cambios nuevos no rompan funcionalidades previas
Smoke Testing	Humo – Previo a releases	TC críticos de cada módulo	Confirmar que el sistema inicia y funciones principales responden

Plantilla de Caso de Prueba

Todos los casos de prueba deberán documentarse siguiendo la plantilla estándar definida a continuación. Esta plantilla garantiza uniformidad en la información recopilada y facilita la revisión por parte del Coordinador de Calidad.

Campo	Descripción / Valor
ID	TC-[número secuencial]: Identificador único del caso de prueba
Nombre	Nombre descriptivo de la funcionalidad que se está probando
Historia de Usuario	Código y nombre de la historia de usuario asociada (ej: HU-001 – Inicio de sesión con Google)
Objetivo	Descripción del comportamiento o criterio de aceptación que se desea validar
Precondiciones	Estado requerido del sistema antes de ejecutar la prueba (usuario registrado, datos previos, etc.)
Datos de entrada	Valores específicos que se utilizarán durante la ejecución (correos, contraseñas, inputs)
Pasos de ejecución	Secuencia numerada de acciones a realizar para ejecutar la prueba
Resultado esperado	Comportamiento que el sistema debe mostrar si la funcionalidad es correcta

Resultado obtenido	Comportamiento real observado durante la ejecución de la prueba
Evidencia	Captura de pantalla, video o registro adjunto que respalde el resultado
Estado	Aprobado / Rechazado / Bloqueado
Ejecutado por	Nombre del responsable que ejecutó el caso de prueba
Fecha de ejecución	Fecha en que se ejecutó la prueba
Observaciones	Notas adicionales, defectos encontrados o referencias a tickets de corrección

Recopilación de Evidencias

Toda ejecución de un caso de prueba deberá ir acompañada de evidencia que respalde el resultado registrado. La siguiente tabla detalla los tipos de evidencia aceptados y la forma en que deben almacenarse en Jira:

Tipo de evidencia	Cuándo se usa	Cómo se almacena en Jira
Captura de pantalla	Resultado visible en UI, mensajes de error, estados del sistema	Adjunto directo al Test Case en el campo "Evidencia"
Video corto (máx. 2 min)	Flujos complejos, comportamiento dinámico, animaciones	Enlace a Google Drive adjunto al Test Case
Log del sistema / consola	Errores de servidor, excepciones, validaciones de backend	Archivo .txt o .log adjunto al Test Case
Resultado de prueba en Jira	Estado final (Aprobado/Rechazado), trazabilidad automática	Registrado automáticamente al marcar el estado del Test Case

Responsables de Ejecución y Aprobación

- La ejecución de los casos de prueba será responsabilidad del Coordinador de Calidad, con apoyo del equipo de desarrollo según la complejidad del sprint.
- Ningún desarrollador podrá aprobar los casos de prueba asociados a su propio trabajo.

- El Coordinador General deberá validar los Test Cycles antes del cierre de cada sprint.
- Los casos de prueba rechazados generarán un ticket de defecto en Jira que deberá resolverse antes del cierre del sprint.

Diseño y Especificaciones de las Pruebas Unitarias

Las pruebas unitarias tienen como objetivo verificar el correcto funcionamiento de los componentes individuales del sistema antes de su integración con otros módulos. Su aplicación permite detectar defectos de forma temprana, reducir retrabajos y asegurar que la lógica principal de la plataforma funcione correctamente desde las primeras etapas del desarrollo.

Para el backend desarrollado en Java se utilizará JUnit 5 como herramienta principal para la creación y ejecución de pruebas unitarias. Esta herramienta se encuentra alineada con el ecosistema Java utilizado en el curso y es una de las opciones estándar para validar métodos, clases, servicios y reglas de negocio dentro de aplicaciones desarrolladas en Java.

Además, se utilizará Mockito como herramienta complementaria para la simulación de dependencias. Esto permitirá probar servicios de forma aislada sin depender directamente de repositorios, bases de datos, servicios externos o integraciones con APIs, facilitando la validación individual de la lógica del sistema.

Para el frontend desarrollado en Angular se utilizarán Jasmine y Karma, herramientas comúnmente integradas por defecto en proyectos Angular. Jasmine permitirá definir los escenarios de prueba y las validaciones esperadas, mientras que Karma se encargará de ejecutar dichas pruebas en un entorno controlado. Estas herramientas se encuentran alineadas con las tecnologías utilizadas en el proyecto y con el entorno de desarrollo definido para el curso, basado en Java para el backend y Angular para el frontend.

Las pruebas unitarias deberán desarrollarse de forma paralela a la implementación de cada funcionalidad. Ninguna historia de usuario podrá considerarse finalizada sin haber validado previamente el comportamiento de los componentes desarrollados mediante las pruebas correspondientes.

Las pruebas unitarias se enfocarán principalmente en los módulos que contienen lógica crítica para el negocio, entre ellos:

- Procesos de autenticación y gestión de usuarios.
- Gestión de ligas, torneos y temporadas.
- Generación y actualización de fixtures y brackets.
- Registro y validación de resultados.
- Gestión de disputas y sanciones.
- Procesamiento de métricas y estadísticas.
- Integraciones con servicios externos.
- Validaciones de reglas de negocio y criterios de elegibilidad.

La ejecución de las pruebas unitarias deberá realizarse antes de solicitar la integración de cambios al repositorio principal. Cada desarrollador será responsable de verificar el correcto funcionamiento de su código y corregir cualquier defecto identificado antes de presentar su trabajo para revisión.

La verificación de que las pruebas unitarias fueron efectivamente realizadas se llevará a cabo durante el proceso de revisión de código y aprobación de Pull Requests. Los Coordinadores de Desarrollo y de Calidad deberán comprobar que la funcionalidad desarrollada cuenta con pruebas unitarias suficientes, que estas fueron ejecutadas correctamente y que no presentan errores antes de autorizar su integración.

Asimismo, el equipo utilizará GitHub Actions para automatizar la ejecución de pruebas cuando sea posible, con el fin de detectar errores de manera temprana y mantener la estabilidad del sistema durante todo el desarrollo.

Como evidencia de cumplimiento, los resultados de las pruebas deberán encontrarse disponibles para su revisión durante el proceso de control de calidad y previo al cierre de cada sprint.

Diseño y Especificaciones de las Pruebas de Rendimiento

Las pruebas de rendimiento tienen como propósito evaluar la capacidad de respuesta, estabilidad y comportamiento general de la plataforma bajo diferentes niveles de carga. Estas pruebas permitirán identificar posibles cuellos de botella, validar el desempeño de los servicios críticos y verificar que el sistema sea capaz de soportar un volumen razonable de usuarios y operaciones concurrentes.

Para la validación funcional de los servicios API se utilizará Postman, herramienta incorporada dentro del entorno de desarrollo utilizado en el curso y empleada para verificar el correcto funcionamiento de los endpoints, las respuestas del sistema, los mecanismos de autenticación y el cumplimiento de los criterios de aceptación asociados a cada servicio.

Sin embargo, para la ejecución de las pruebas de rendimiento y carga se utilizará Apache JMeter. Esta herramienta fue seleccionada porque permite simular múltiples usuarios concurrentes, generar carga controlada sobre los servicios del sistema y recopilar métricas especializadas de desempeño. Además, se encuentra alineada con las tecnologías utilizadas en el proyecto y con el entorno de desarrollo definido para el curso, ya que permite probar endpoints expuestos por el backend desarrollado en Java y simular múltiples solicitudes concurrentes hacia el backend del sistema.

La combinación de Postman y Apache JMeter permite cubrir dos objetivos distintos dentro del aseguramiento de la calidad: Postman se utilizará para validar que los servicios funcionen correctamente desde el punto de vista funcional, mientras que JMeter se utilizará para determinar cómo se comporta el sistema cuando recibe múltiples solicitudes simultáneas o se encuentra sometido a condiciones de carga más exigentes.

Apache JMeter permitirá crear escenarios de prueba reproducibles, simular usuarios concurrentes, enviar solicitudes HTTP a los servicios del sistema y recopilar métricas relacionadas con el desempeño de la plataforma. Su uso resulta adecuado para Brakket debido a que el sistema incluye módulos que podrían recibir múltiples solicitudes simultáneas, como autenticación, consulta de torneos, generación de fixtures, registro de resultados, visualización de estadísticas y procesamiento de datos de transmisiones.

El alcance de las pruebas de rendimiento se centrará principalmente en los módulos que presentan mayor impacto sobre el funcionamiento de la plataforma, entre ellos:

- Procesos de autenticación e inicio de sesión.
- Creación y consulta de ligas, torneos y temporadas.
- Generación y actualización de fixtures y brackets.
- Registro y consulta de resultados.
- Consultas de estadísticas e historial competitivo.
- Captura y procesamiento de métricas provenientes de transmisiones.
- Visualización de paneles administrativos y reportes.

Las pruebas buscarán medir, entre otros aspectos:

- Tiempo promedio de respuesta.
- Tiempo máximo de respuesta.
- Cantidad de solicitudes procesadas por unidad de tiempo.
- Porcentaje de solicitudes fallidas.
- Comportamiento del sistema bajo carga concurrente.
- Estabilidad general durante períodos prolongados de ejecución.

Dentro de Apache JMeter se utilizarán principalmente los siguientes componentes:

- Thread Group: permitirá definir la cantidad de usuarios virtuales, el tiempo de subida de carga y el número de repeticiones de la prueba.
- HTTP Request: permitirá configurar las solicitudes hacia los endpoints del backend que serán evaluados.
- HTTP Header Manager: permitirá agregar encabezados necesarios para las solicitudes, como tokens de autenticación o tipo de contenido.

- View Results Tree: permitirá revisar de forma detallada las solicitudes ejecutadas, las respuestas recibidas y los posibles errores.
- Summary Report: permitirá analizar métricas generales como tiempo promedio, mínimo, máximo, porcentaje de error y cantidad de solicitudes procesadas.
- Aggregate Report: permitirá comparar el comportamiento de distintos endpoints y detectar cuáles presentan mayor tiempo de respuesta o mayor tasa de error.

Las pruebas de rendimiento se realizarán en diferentes momentos del proyecto para garantizar un monitoreo continuo del desempeño:

- Al finalizar el desarrollo de los módulos principales.
- Durante las etapas de integración de funcionalidades.
- Antes de la entrega final del proyecto.
- Cuando se incorporen cambios significativos que puedan afectar el rendimiento del sistema.

Para cada ejecución se definirá un escenario de prueba que contemple la cantidad de usuarios simulados, las operaciones que realizarán y la duración de la prueba. Los resultados obtenidos serán comparados con los objetivos de desempeño establecidos por el equipo para identificar posibles oportunidades de optimización.

Los reportes generados por JMeter serán almacenados como evidencia del proceso de aseguramiento de la calidad y servirán como insumo para la toma de decisiones relacionadas con mejoras de rendimiento, optimización de consultas y ajustes en la arquitectura del sistema.

La revisión de los resultados será responsabilidad conjunta del equipo de desarrollo y del Coordinador de Calidad, quienes evaluarán si el comportamiento observado cumple con los niveles de desempeño esperados antes de considerar una funcionalidad como apta para su liberación.

Definition of Done

La Definition of Done (DoD) representa el acuerdo de calidad establecido por el equipo de trabajo para determinar cuándo una historia de usuario, requerimiento o incremento del producto puede considerarse realmente terminado. Su propósito es garantizar que todos los integrantes compartan una misma comprensión sobre el significado de "completado" y que cada funcionalidad entregada cumpla con los estándares mínimos de calidad definidos para el proyecto.

La Definition of Done se aplica a todas las historias de usuario desarrolladas durante el proyecto y deberá cumplirse en su totalidad antes de que una historia pueda pasar al estado "Hecho" dentro de Jira.

Checklist de Definition of Done

Criterio	Cumple
Los criterios de aceptación de la historia de usuario fueron implementados completamente	☐
La funcionalidad cumple con los requerimientos definidos en la ERS	☐
El código cumple con los estándares de codificación establecidos por el equipo	☐
El código fue revisado por al menos un integrante distinto al desarrollador responsable	☐
Las observaciones identificadas durante la revisión fueron atendidas y corregidas	☐
Las pruebas unitarias fueron ejecutadas satisfactoriamente	☐
Las pruebas funcionales asociadas fueron ejecutadas y aprobadas	☐
No existen defectos críticos o bloqueantes pendientes relacionados con la funcionalidad	☐
La funcionalidad se integró correctamente con el resto del sistema	☐

La documentación técnica afectada fue actualizada cuando corresponde ☐

Las evidencias de prueba fueron registradas y almacenadas según el proceso de calidad definido ☐

La funcionalidad fue validada por el Coordinador de Calidad o responsable asignado ☐

Acuerdos Generales del Equipo

Además de los criterios anteriores, el equipo establece los siguientes acuerdos:

- Ningún integrante podrá aprobar su propio trabajo.
- Toda funcionalidad deberá encontrarse respaldada en el repositorio oficial del proyecto antes del cierre del sprint.
- Los defectos identificados durante las actividades de prueba deberán corregirse antes de considerar una historia como terminada.
- Una historia de usuario que no cumpla cualquiera de los criterios definidos en esta Definition of Done no podrá ser considerada finalizada y deberá permanecer en el flujo de trabajo hasta completar los requisitos pendientes.
- La Definition of Done será revisada periódicamente durante las retrospectivas del proyecto y podrá ajustarse cuando el equipo identifique oportunidades para fortalecer los estándares de calidad del producto.

El cumplimiento de esta Definition of Done garantiza que cada incremento entregado mantenga un nivel uniforme de calidad, reduzca la incorporación de defectos al sistema y facilite la evolución y mantenimiento futuro de la plataforma.

Plan de Gestión de Riesgos

El presente apartado define la estrategia que el equipo Coffee&Commits seguirá para gestionar los riesgos del proyecto Brakket a lo largo de todo su ciclo de vida. Su propósito es anticipar las situaciones que podrían afectar el cumplimiento de los objetivos del proyecto en cuanto a alcance, tiempo, calidad y configuración y definir, para cada una de ellas, las acciones de mitigación y de contingencia correspondientes.

La gestión de riesgos se aborda mediante un análisis cualitativo, en el cual cada riesgo identificado se evalúa según dos dimensiones: la probabilidad de que ocurra y el impacto que tendría sobre el proyecto en caso de materializarse. A partir de la combinación de ambas dimensiones se ubica cada riesgo dentro de una matriz de probabilidad e impacto, lo que permite categorizarlo según su severidad (bajo, medio-bajo, medio-alto o alto) y priorizar la atención sobre los más críticos.

Los riesgos se agrupan en cuatro áreas: equipo y organización, tiempo y carga académica, tecnología y servicios externos, y presupuesto y pruebas. Para todos los riesgos clasificados como medio-bajo (amarillo), medio-alto (mostaza) y alto (rojo) se define un plan de mitigación, orientado a reducir la probabilidad de ocurrencia o el impacto del riesgo, y un plan de contingencia, que establece las medidas a tomar en caso de que el riesgo llegue a materializarse. Los riesgos de severidad baja (verde) se mantienen únicamente bajo monitoreo.

El registro de riesgos no es un artefacto estático: se mantiene disponible para el equipo y se revisa de forma periódica durante las reuniones de seguimiento de cada sprint, incorporando los nuevos riesgos que puedan surgir durante el desarrollo para someterlos al mismo proceso de análisis y respuesta.

Tabla de Riesgos

La siguiente tabla constituye el registro de riesgos del proyecto. En ella se listan los riesgos identificados, organizados por área, indicando para cada uno su descripción, la probabilidad y el impacto estimados según las escalas definidas anteriormente, y la severidad resultante de ubicarlo en la matriz de probabilidad e impacto.

ID	Riesgo	Descripción	Probabilidad	Impacto	Severidad
Equipo y organización					
R-01	Ausencia o abandono de un integrante	La salida o reducción de participación de un miembro obliga a redistribuir su carga entre el resto del equipo.	Posible (2)	Peligroso (D)	Medio-bajo
R-02	Concentración de la carga técnica	La mayor parte del trabajo técnico recae sobre pocos integrantes, generando un punto único de dependencia.	Ocasional (3)	Moderado (C)	Medio-bajo
R-03	Conflicto de criterios entre	Diferencias de criterio entre quienes supervisan la calidad	Posible (2)	Menor (B)	Bajo

ID	Riesgo	Descripción	Probabilidad	Impacto	Severidad
	Coordinadores de Calidad	generan retrabajos o retrasos en las decisiones.			
R-04	Deuda técnica acumulada	La presión por las entregas lleva a soluciones apresuradas que deterioran la calidad del código y dificultan su mantenimiento.	Probable (4)	Moderado (C)	Medio-alto
Tiempo y carga académica					
R-05	Alcance superior al tiempo disponible	El alcance planteado excede lo implementable dentro del cuatrimestre, comprometiendo la entrega de un producto funcional completo.	Probable (4)	Peligroso (D)	Alto
R-06	Interrupciones por otros cursos	Los exámenes y entregas de otras materias reducen las horas efectivas de desarrollo, sobre todo cuando coinciden en el tiempo.	Frecuente (5)	Menor (B)	Medio-alto
Tecnología y servicios externos					
R-07	Fallos o cambios en la API de Twitch	Las condiciones de uso, límites o disponibilidad de la API pública de Twitch cambian, afectando la transmisión y la captura de datos.	Posible (2)	Peligroso (D)	Medio-bajo
R-08	Incumplimiento de los Términos de Servicio de Twitch	El almacenamiento del chat capturado entra en conflicto con las políticas de la plataforma, obligando a replantear su manejo.	Ocasional (3)	Moderado (C)	Medio-bajo
R-09	Complejidad del análisis de sentimiento con IA	La implementación del análisis de sentimiento supera la capacidad técnica o la experiencia del equipo, afectando los tiempos de desarrollo.	Probable (4)	Moderado (C)	Medio-alto
R-10	Algoritmos de fixtures más complejos de lo estimado	La generación automática de enfrentamientos para los distintos formatos resulta más difícil de lo previsto, sobre todo en los casos especiales.	Ocasional (3)	Peligroso (D)	Medio-alto
Presupuesto y pruebas					
R-11	Costos de servicios cloud por encima	El uso de servicios de alojamiento, almacenamiento o herramientas de pago supera	Posible (2)	Moderado (C)	Medio-bajo

ID	Riesgo	Descripción	Probabilidad	Impacto	Severidad
	del presupuesto	los recursos económicos disponibles del equipo.			
R-12	Falta de usuarios reales para pruebas	La ausencia de una base de usuarios real dificulta probar el sistema en condiciones realistas, en especial los módulos de audiencia y transmisión.	Probable (4)	Menor (B)	Medio-bajo

Tabla de Probabilidad e Impacto

La matriz de probabilidad e impacto permite ubicar visualmente cada riesgo en el cuadrante que resulta del cruce entre su probabilidad (filas) y su impacto (columnas). El color de cada celda indica la severidad del riesgo: verde representa un nivel bajo, amarillo un nivel medio-bajo, mostaza un nivel medio-alto y rojo un nivel alto. Los riesgos ubicados en las zonas mostaza y roja son los que requieren mayor atención y seguimiento.

Probabilidad / Impacto	Insignificante (A)	Menor (B)	Moderado (C)	Peligroso (D)	Catastrófico (E)
Frecuente (5)		R-06			
Probable (4)		R-12	R-04, R-09	R-05	
Ocasional (3)			R-02, R-08	R-10	
Posible (2)		R-03	R-11	R-01, R-07	
Improbable (1)					

Leyenda de severidad:

Bajo	Medio-bajo	Medio-alto	Alto
-------------	-------------------	-------------------	-------------

Mitigaciones y Contingencias

Para cada riesgo clasificado con severidad medio-baja, medio-alta o alta se definen dos puntos de control: un plan de mitigación, orientado a reducir la probabilidad de que el riesgo ocurra o su impacto, y un plan de contingencia, que establece las medidas a tomar en caso de que el riesgo llegue a materializarse. Los riesgos de severidad baja, como el conflicto de criterios entre Coordinadores de Calidad (R-03), se mantienen únicamente bajo monitoreo y no requieren planes formales de respuesta.

ID	Riesgo	Severidad	Plan de Mitigación	Plan de Contingencia
R-01	Ausencia o abandono de un integrante	Medio-bajo	Documentar el trabajo de cada integrante en el repositorio y respaldar continuamente los avances en GitHub; promover comunicación constante y un reparto equilibrado de tareas para evitar el conocimiento aislado.	Redistribuir las tareas del integrante ausente entre el resto del equipo, repriorizar el backlog reduciendo el alcance de los módulos no esenciales y notificar la situación a los profesores del curso.
R-02	Concentración de la carga técnica	Medio-bajo	Distribuir los módulos según las fortalezas de cada integrante pero rotando responsabilidades; realizar revisiones cruzadas de código mediante Pull Requests para difundir el conocimiento del sistema.	Reasignar temporalmente integrantes hacia los módulos sobrecargados y ajustar la meta del sprint afectado.
R-04	Deuda técnica acumulada	Medio-alto	Aplicar los estándares de codificación y la Definition of Done definidos; exigir revisión de código antes de cada merge y reservar tiempo de refactorización al cierre de cada sprint.	Registrar la deuda técnica como tickets en Jira y atenderlos en un sprint de estabilización antes de la entrega final.
R-05	Alcance superior al tiempo disponible	Alto	Priorizar el backlog en torno a un MVP claramente definido; planificar por sprints de dos semanas con revisión del avance real y mantener las funcionalidades opcionales claramente separadas de las esenciales.	Recortar funcionalidades no esenciales (por ejemplo, progresión y personalización, o parte del análisis con IA) preservando el flujo principal del sistema, y comunicar el ajuste de alcance a los profesores.
R-06	Interrupción por otros cursos	Medio-alto	Elaborar el cronograma considerando las fechas de exámenes y entregas de otros cursos, adelantando trabajo durante las semanas de menor carga académica.	Redistribuir las tareas hacia los integrantes con menor carga esa semana y ajustar la meta del sprint según la disponibilidad real.

ID	Riesgo	Severidad	Plan de Mitigación	Plan de Contingencia
R-07	Fallos o cambios en la API de Twitch	Medio-bajo	Encapsular la integración con Twitch en una capa de servicios desacoplada del resto del sistema, revisar la documentación oficial y trabajar dentro de los límites del nivel gratuito de la API.	Utilizar datos simulados (mocks) de audiencia y chat para no bloquear el desarrollo ni la demostración, conforme a la premisa de pruebas con datos simulados.
R-08	Incumplimiento de los Términos de Servicio de Twitch	Medio-bajo	Revisar los Términos de Servicio antes de implementar la captura del chat y almacenar únicamente datos agregados y los resultados del análisis, evitando guardar el contenido íntegro de forma indebida.	Replantear el esquema de almacenamiento para conservar solo métricas agregadas o el resultado del análisis de sentimiento, eliminando cualquier dato sensible.
R-09	Complejidad del análisis de sentimiento con IA	Medio-alto	Apoyarse en servicios o APIs de IA de terceros ya entrenados en lugar de modelos propios, realizar una prueba de concepto temprana y mantener el alcance del módulo simple (positivo, negativo o neutral).	Simplificar el módulo a un análisis basado en reglas o léxico, o reducirlo a una clasificación básica validada con datos de prueba.
R-10	Algoritmos de fixtures más complejos de lo estimado	Medio-alto	Implementar primero los formatos más simples (eliminación simple y round robin) y abordar los complejos (doble eliminación y suizo) de forma incremental con pruebas unitarias; investigar librerías existentes.	Limitar los formatos soportados a los ya implementados y validados, documentando los pendientes como mejoras futuras del sistema.
R-11	Costos de servicios cloud por encima del presupuesto	Medio-bajo	Utilizar servicios dentro de su nivel gratuito, estimar el consumo antes de desplegar y preferir opciones de alojamiento sin costo para los entornos de prueba.	Migrar a alternativas gratuitas equivalentes o ejecutar la demostración en un entorno local en caso de superar los límites disponibles.
R-12	Falta de usuarios reales para pruebas	Medio-bajo	Generar datos de prueba representativos (equipos, jugadores, torneos y transmisiones simuladas) y diseñar casos de prueba que cubran los flujos principales sin requerir usuarios reales.	Realizar la validación y la demostración final con datos simulados y pruebas cruzadas entre los propios integrantes del equipo.

Plan de Administración de la Configuración

Descripción de las Herramientas

Para la gestión de la configuración del proyecto se utilizarán las siguientes herramientas:

Git

Sistema de control de versiones distribuido que permite registrar, administrar y rastrear todos los cambios realizados sobre el código fuente y la documentación del proyecto.

Proveedor: Git Project.

GitHub

Plataforma basada en la nube utilizada para alojar el repositorio del proyecto, gestionar ramas, solicitudes de cambio (Pull Requests), revisiones de código y control de acceso de los miembros del equipo.

Proveedor: GitHub Inc.

GitHub Actions

Herramienta integrada en GitHub utilizada para automatizar procesos de integración continua (CI), validaciones y futuras tareas de despliegue.

Proveedor: GitHub Inc.

Google Drive

Herramienta utilizada para almacenar documentos del proyecto, entregables, minutas y demás artefactos administrativos.

Proveedor: Google LLC.

Administración del Repositorio

Estructura del Repositorio

```
brakket/  
├── frontend/  
├── backend/
```

```
|— docs/  
|— README.md  
|— .gitignore
```

Estrategia de Ramas

Se utilizará una estrategia basada en ramas de integración:

- main: contiene únicamente versiones estables y aprobadas del sistema.
- develop: rama principal de integración utilizada durante el desarrollo.
- feature/*: utilizada para el desarrollo de nuevas funcionalidades.
- bugfix/*: utilizada para corrección de errores.
- hotfix/*: utilizada para correcciones urgentes.

Flujo de Trabajo

1. Cada desarrollador crea una rama a partir de develop.
2. El desarrollo de cada tarea se realiza exclusivamente en dicha rama.
3. Al finalizar la implementación, el desarrollador crea un Pull Request hacia develop.
4. El Pull Request debe ser revisado por al menos un Coordinador de Desarrollo y un Coordinador de Calidad.
5. Una vez aprobado, el cambio se integra mediante Merge commit.
6. Al finalizar cada sprint, los cambios validados en develop serán incorporados a main

Reglas del Repositorio

Reglas Generales

- No se permite realizar cambios directamente sobre la rama main.
- Todo desarrollo debe realizarse en ramas específicas de trabajo.
- Todo cambio debe integrarse mediante Pull Request.
- No se permitirá realizar merges sin revisión previa.
- Los conflictos de integración deberán resolverse antes de solicitar aprobación.

Convención de Commits

Los commits deberán seguir el estándar Conventional Commits:

feat: agregar integración con Twitch

fix: corregir generación de brackets

docs: actualizar documentación técnica

refactor: simplificar módulo de autenticación

Frecuencia de Actualización

Con el fin de garantizar la trazabilidad y el respaldo de los cambios realizados, cada integrante deberá registrar periódicamente sus avances mediante commits descriptivos. Los cambios desarrollados durante cada sprint deberán sincronizarse regularmente con el repositorio central, evitando la acumulación excesiva de modificaciones locales y facilitando la integración entre los miembros del equipo. Asimismo, toda funcionalidad concluida deberá encontrarse respaldada en el repositorio antes del cierre del sprint correspondiente.

Contenido de los Commits

Los commits deberán contener cambios relacionados con una única funcionalidad, corrección o tarea específica para facilitar el seguimiento y la auditoría del proyecto.

Revisiones de Código

Antes de integrarse a la rama develop, cada Pull Request deberá ser revisado y aprobado por los responsables correspondientes. Las observaciones realizadas durante la revisión deberán ser atendidas antes de la integración final.

Plan de Estándares

El presente plan define los estándares de desarrollo que el equipo seguirá a lo largo de todo el proyecto **Brakket**, con el fin de garantizar la consistencia, la legibilidad y la mantenibilidad tanto del código como de la interfaz y la base de datos. Establecer estos lineamientos desde el inicio permite que todos los integrantes trabajen bajo criterios comunes, facilitando la colaboración en paralelo, la revisión del trabajo entre compañeros y la incorporación de cualquier miembro a una parte del proyecto que no haya desarrollado directamente.

Los estándares aquí definidos abarcan los siguientes aspectos: los estándares de color y de controles de interfaz gráfica, que determinan la apariencia y el comportamiento visual de la plataforma; los estándares de nomenclatura de código y de base de datos, que unifican la forma en que se nombran los distintos elementos del sistema; y los estándares de documentación, de carácter obligatorio, que aseguran que tanto el código como los objetos de la base de datos queden debidamente documentados. Estos lineamientos se detallan en los apartados de Estándares de Interfaz Gráfica y Estándares de Codificación que se presentan a continuación.

Estándares de Interfaz Gráfica

Esta sección define los lineamientos visuales que regirán el diseño de la interfaz de **Brakket**, abarcando la paleta de colores, la tipografía, la nomenclatura de los controles y la apariencia y comportamiento de los componentes que se utilizarán a lo largo de toda la plataforma. El objetivo es garantizar una experiencia de usuario coherente y reconocible en todas las pantallas del producto.

Brakket adopta una identidad visual moderna y tecnológica, basada en una interfaz oscura con el azul como color de marca y acentos luminosos que resaltan los elementos clave. Este enfoque busca transmitir la energía del mundo competitivo de los esports, a la vez que reduce la fatiga visual durante sesiones de uso prolongadas.

Paleta de Colores

La paleta se construye sobre fondos oscuros, una familia de azules como color principal y un acento de alta luminosidad para destacar acciones e información relevante.

Uso	Color	Código
Fondo Principal	Negro Azulado	#0A0E1A
Fondo de Superficies (cards, paneles)	Azul muy Oscuro	#131A2E
Color Primario / Marca	Azul Brakket	#2563EB
Color Primario (variante clara)	Azul Brillante	#3B82F6
Acento / Resaltado	Cian Neón	#22D3EE
Texto Principal	Blanco Suave	#F1F5F9
Texto Secundario	Gris Azulado	#94A3B8
Bordes y Divisores	Gris Oscuro Azulado	#1E293B
Estado de Éxito	Verde Neón	#22C55E
Estado de Advertencia	Ámbar	#F59E0B
Estado de Error	Rojo	#EF4444

El azul primario se reserva para acciones principales, enlaces y elementos de marca. El cian neón funciona como acento para destacar elementos interactivos, estados activos o datos importantes (por ejemplo, un marcador en vivo o una métrica destacada), y debe usarse con moderación para no perder su efecto. Los fondos oscuros predominan en toda la plataforma, y el texto se maneja en blanco suave sobre ellos para garantizar contraste y legibilidad.

Tipografía

Se define una tipografía principal de tipo sans-serif de apariencia moderna, que garantice buena legibilidad sobre fondos oscuros. Se establecen jerarquías claras: un tamaño mayor y peso bold para títulos, un tamaño intermedio para subtítulos y un tamaño base para el cuerpo de texto, manteniendo proporciones consistentes en toda la plataforma. Para datos numéricos destacados, como marcadores o estadísticas, puede emplearse un peso más marcado que refuerce el carácter competitivo de la interfaz.

Nomenclatura de Controles

Para mantener consistencia en el desarrollo, todos los controles de la interfaz seguirán una convención de nombres uniforme, compuesta por un prefijo que identifica el tipo de control

seguido de un nombre descriptivo en formato camelCase. Esto facilita la identificación de cada elemento tanto en el código como en las herramientas de diseño.

Tipo de Control	Prefijo	Ejemplo
Botón	btn	btnCrearTorneo
Campo de texto	txt	txtNombreEquipo
Lista Desplegable	ddl	ddlSeleccionarJuego
Tabla	tbl	tblPosiciones
Lista	lst	lstParticipantes
Casilla de Verificación	chk	chkAceptarReglas
Botón de Opción	rdb	rdbFormatoTorneo
Etiqueta	lbl	lblEstadoPartida

Componentes de la Interfaz

A continuación se definen los componentes estándar que se utilizarán de forma consistente en toda la plataforma:

- **Botones:** se distinguen tres variantes. El botón primario, con fondo en azul de marca y texto claro, se usa para la acción principal de cada pantalla. El botón secundario, con borde azul y fondo transparente sobre el fondo oscuro, para acciones alternativas. El botón deshabilitado se muestra en gris azulado sin interacción. Todos presentan esquinas levemente redondeadas y un efecto luminoso sutil al pasar el cursor.
- **Campos de texto:** fondo de superficie oscuro con borde en gris azulado en estado normal, que cambia a azul brillante o cian al estar enfocados. Incluyen una etiqueta superior y, cuando corresponde, un mensaje de validación en color de estado debajo del campo.
- **Listas desplegables:** mantienen el mismo estilo visual que los campos de texto, con un ícono de flecha a la derecha que indica su capacidad de despliegue.

- **Tablas:** encabezado con fondo de superficie y texto destacado, filas con separadores sutiles en tono oscuro y resaltado de fila en azul translúcido al pasar el cursor. Se utilizan para tablas de posiciones, listados de equipos y estadísticas.
- **Casillas de verificación y botones de opción:** adoptan el color primario o el acento neón cuando están seleccionados, con un tamaño suficiente para facilitar su interacción.
- **Notificaciones y mensajes:** utilizan los colores de estado definidos en la paleta, mostrándose sobre superficies oscuras con un ícono que refuerza el tipo de mensaje (éxito, error, advertencia o información).
- **Tarjetas (cards):** contenedores con fondo de superficie azul oscuro, esquinas redondeadas y un borde o sombra sutil que las separa del fondo principal. Se utilizan para agrupar información como perfiles de jugador, equipos, torneos o datos de transmisión.

Estándares de Codificación

Esta sección define las convenciones de codificación que el equipo seguirá durante el desarrollo de **Brakket**, abarcando la base de datos, el desarrollo de frontend con Angular y JavaScript, y el desarrollo de backend con Java. El objetivo es mantener un código uniforme, legible y mantenible, de modo que cualquier integrante pueda comprender y trabajar sobre el código de otro. La documentación del código mediante JavaDoc es de carácter obligatorio.

Estándares Generales

- El código, los nombres de variables, funciones y comentarios se escribirán de forma clara y descriptiva, evitando abreviaturas ambiguas.
- Se mantendrá una indentación consistente y un único estilo de formato en todo el proyecto.
- No se dejará código comentado ni sin uso en las versiones finales.

- Los nombres serán significativos y describirán la intención del elemento, no su tipo.

Estándares de Base de Datos

- Los nombres de tablas se escribirán en plural y en formato snake_case (por ejemplo, usuarios, equipos, torneos_jugados).
- Los nombres de columnas se escribirán en snake_case y en singular (por ejemplo, fecha_creacion, id_equipo).
- Cada tabla contará con una clave primaria identificada de forma consistente (por ejemplo, id o id_<entidad>).
- Las claves foráneas seguirán una nomenclatura que refleje la tabla referenciada (por ejemplo, id_usuario).
- Los objetos de la base de datos (tablas, vistas, procedimientos) deberán estar documentados, describiendo su propósito y el de sus campos.

Estándares de Código en Java (Backend)

- Las clases se nombrarán en PascalCase (por ejemplo, TorneoService), y los métodos y variables en camelCase (por ejemplo, crearTorneo, listaParticipantes).
- Las constantes se escribirán en mayúsculas con guiones bajos (por ejemplo, MAX_JUGADORES).
- Cada clase, método público y atributo relevante deberá documentarse mediante JavaDoc, de manera obligatoria, indicando su propósito, parámetros, valores de retorno y excepciones cuando corresponda.
- Se seguirá una organización de paquetes coherente que separe responsabilidades (por ejemplo, controladores, servicios, modelos y repositorios).

Estándares de Código en Angular y Javascript (Frontend)

- Los componentes se nombrarán de forma descriptiva siguiendo la convención de Angular (por ejemplo, torneo-lista.component.ts).
- Las clases y componentes se escribirán en PascalCase, mientras que las variables, funciones y métodos en camelCase.
- Los nombres de archivos seguirán el formato kebab-case, según la convención estándar de Angular.
- Se favorecerá el uso de tipado cuando esté disponible, evitando el uso innecesario de tipos genéricos o ambiguos.
- Las funciones y métodos relevantes se documentarán mediante comentarios que expliquen su propósito, parámetros y valor de retorno.
- Se mantendrá una separación clara entre la lógica, la presentación y los estilos de cada componente.

Estándares de Documentación

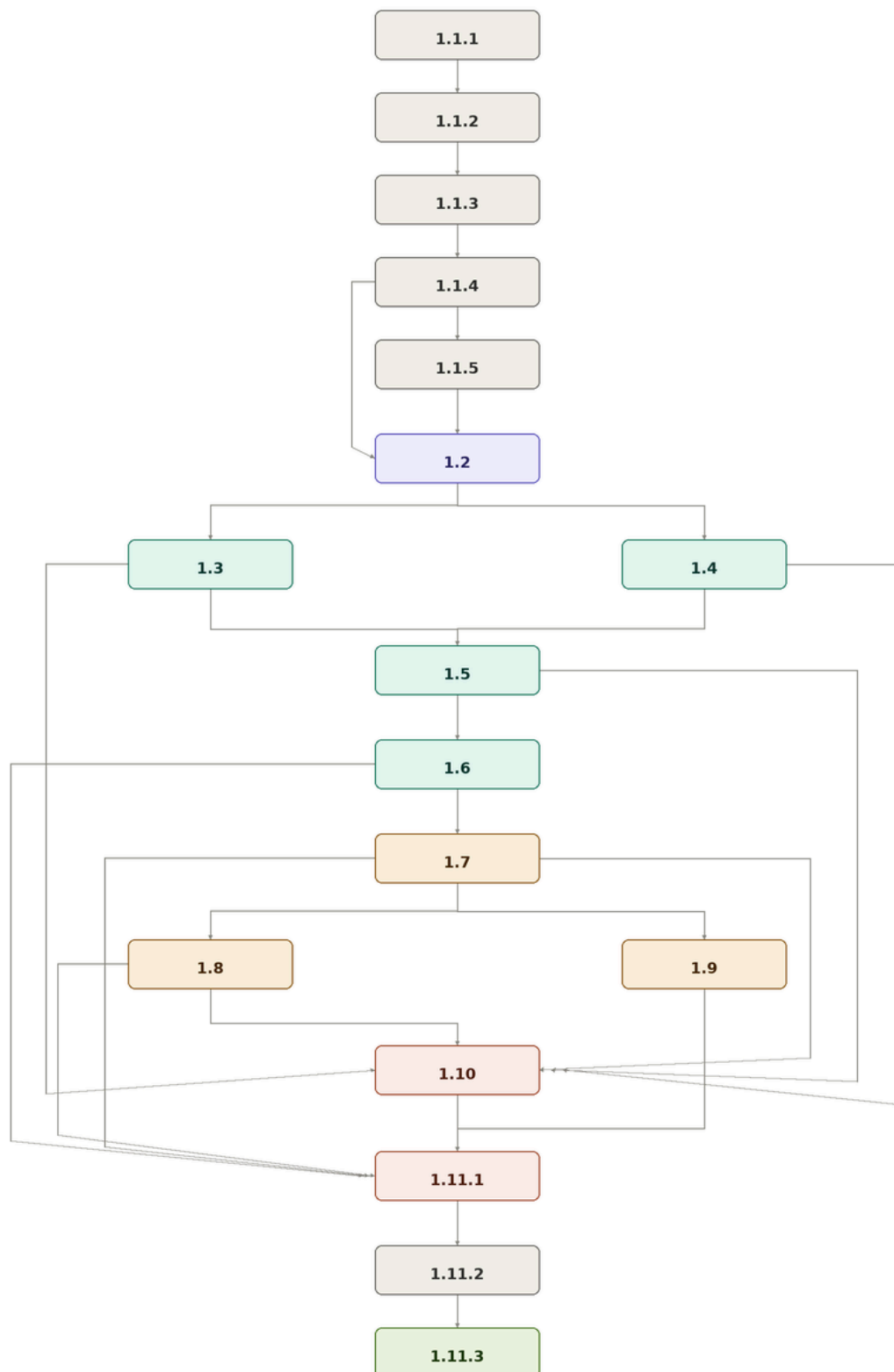
- Todo el código deberá estar documentado de forma que su propósito sea comprensible sin necesidad de recurrir a su autor.
- En el caso de Java, la documentación seguirá obligatoriamente el formato JavaDoc.
- En el caso de Angular y JavaScript, se documentarán las funciones, componentes y módulos siguiendo un formato de comentarios consistente en todo el proyecto.
- Los objetos de la base de datos se documentarán describiendo su función dentro del modelo de datos.

Plan de Administración del Tiempo

Estructura de desglose de trabajo (WBS)

[Excalidraw Whiteboard](#)

Diagrama de Red



Herramienta de Gestión de Proyectos

Flujo de trabajo para historias de usuario

Cada historia de usuario seguirá el siguiente flujo dentro de Jira:

Estado	Descripción
Por hacer	La historia fue creada y está en el backlog, pendiente de asignación
En progreso	Un desarrollador tomó la historia y está trabajando en ella activamente
En revisión	El desarrollo está completo y se envió para revisión de código por otro miembro
En pruebas	El Coordinador de Calidad ejecuta las pruebas funcionales correspondientes
Bloqueado	La historia no puede avanzar por una dependencia externa o un impedimento técnico
Hecho	La historia pasó todas las pruebas y fue aprobada por el Coordinador General y el de Calidad

Una historia de usuario sólo puede marcarse como **Hecho** si cumple con los criterios de aceptación definidos en la ERS y fue validada por al menos uno de los Coordinadores de Calidad o el General. Ningún miembro puede aprobar su propio trabajo.

Las historias se organizan en sprints de dos semanas, con una reunión de planificación al inicio y una revisión breve al final de cada sprint para ajustar el backlog según el avance real del semestre.

Lista de Verificación

Elemento a Revisar	Cumple	No cumple	Fecha de Revisión
Introducción	(x)		07/06/2026
Propósito del documento	(x)		07/06/2026
Descripción general del sistema	(x)		07/06/2026
Estructura del documento	(x)		07/06/2026
Definiciones,acrónimos y abreviatura	(x)		07/06/2026
Plan de administración de calidad	(x)		07/06/2026
Formato y detalle de listas de verificación	(x)		07/06/2026
Diseño y especificaciones de los casos de prueba	(x)		07/06/2026
Herramienta de gestión	(x)		07/06/2026
Jerarquía de pruebas	(x)		07/06/2026
Plantilla de casos de prueba	(x)		07/06/2026
Recopilación de evidencia	(x)		07/06/2026
Responsables de ejecución y aprobación	(x)		07/06/2026
Diseño y especificaciones de las pruebas unitarias	(x)		07/06/2026
Diseño y especificaciones de las pruebas de rendimiento	(x)		07/06/2026
Definition of done	(x)		07/06/2026
Plan de gestión de riesgos	(x)		07/06/2026
Tabla de riesgos	(x)		07/06/2026
Tabla de probabilidad e impacto	(x)		07/06/2026
Mitigaciones y contingencias	(x)		07/06/2026
Plan de administración de la configuración	(x)		07/06/2026

Descripción de las herramientas	(x)		07/06/2026
Git	(x)		07/06/2026
Github	(x)		07/06/2026
Github actions	(x)		07/06/2026
Google drive	(x)		07/06/2026
Administración del repositorio	(x)		07/06/2026
Estructura del repositorio	(x)		07/06/2026
Estructura de ramas	(x)		07/06/2026
Flujo de trabajo	(x)		07/06/2026
Reglas del repositorio	(x)		07/06/2026
Reglas generales	(x)		07/06/2026
Convención de commits	(x)		07/06/2026
Frecuencia de actualización	(x)		07/06/2026
Contenido de los commits	(x)		07/06/2026
Revisiones de código	(x)		07/06/2026
Plan de estándares	(x)		07/06/2026
Estandares de interfaz grafica	(x)		07/06/2026
Paleta de colores	(x)		07/06/2026
Tipografía	(x)		07/06/2026
Nomenclatura de controles	(x)		07/06/2026
Componentes de interfaz	(x)		07/06/2026
Estandares de codificación	(x)		07/06/2026
Estándares generales	(x)		07/06/2026
Estándares de base de datos	(x)		07/06/2026
Estándares de código en java	(x)		07/06/2026
Estándares de código en angular y javascrip	(x)		07/06/2026
Estándares de documentación	(x)		07/06/2026

Plan administración de tiempo	(x)		07/06/2026
Estructura y desglose de trabajo (wbs)	(x)		07/06/2026
Diagrama de red	(x)		07/06/2026
Herramienta de gestión de proyectos	(x)		07/06/2026
Lista de verificación	(x)		07/06/2026

Observaciones / Notas de Incumplimiento: La revisión inició el viernes 5 de junio de 2026, sesión en la cual se entregaron las observaciones de control al equipo de desarrollo. Se acordó que los ajustes debían estar listos para la siguiente inspección. El día 07/06/2026 se ejecutó la revisión final del documento, constatando la corrección de los hallazgos, por lo que el entregable ha sido aprobado.

Dictamen de Calidad: ☒ **Aprobado para Entrega** ☐ Requiere Correcciones

Firma Gestión de Calidad

Fecha de Finalización: 07/06/2026